

NAME: SHARMIN SANDIP TADVI

PRN: 202501110083

CLASS: AL1/A14

practical 3

3.1.1

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim
- The shape of the array using shape
- The total number of elements in the array using size

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, and , representing the number of rows and the number of columns.
- The next lines contain space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use reshape() function to reshape the input array with the specified number of rows and columns.

```
import numpy as np rows,cols =
```

```
map(int,input().split()) elements =
```

```
[] for _ in range(rows) :
```

```
elements.extend(map(int,input().split()))
```

```
array = np.array(elements).reshape(rows,cols)
```

```
print(array) print(array.ndim)
```

```
print(array.shape) print(array.size) output :
```

The screenshot displays a coding platform's test results interface. At the top, there are two summary boxes: 'Average time' showing '0.011 s' and '11.33 ms', and 'Maximum time' showing '0.022 s' and '22.00 ms'. To the right, two green status boxes indicate '2 out of 2 shown test case(s) passed' and '10 out of 10 hidden test case(s) passed'. Below these, a section for 'Test case 1' (22 ms) is expanded, showing a comparison between 'Expected output' and 'Actual output'. The outputs are identical, consisting of four lines of space-separated numbers (3 4, 1 2 3 4, 5 6 7 8, 9 10 11 12) and two lines of NumPy array representations. At the bottom, there are tabs for 'Terminal' and 'Test cases'.

Test case 1 (22 ms)	
Expected output	Actual output
3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
9 10 11 12	9 10 11 12
[[1. 2. 3. 4.]	[[1. 2. 3. 4.]
[5. 6. 7. 8.]	[5. 6. 7. 8.]

3.2.1

The given code takes two matrices, , and , as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition ()**
2. **Subtraction ()**
3. **Element-wise Multiplication ()**
4. **Matrix Multiplication ()**
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for , each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for , each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

code :

```
import numpy as np
```

```
#Input matrices print("Enter Matrix A:") matrix_a =  
np.array([list(map(int, input().split())) for i in range(3)])  
  
print("Enter Matrix B:") matrix_b = np.array([list(map(int,  
input().split())) for i in range(3)])
```

```
#Addition addition_result = matrix_a  
+matrix_b print("Addition (A + B):")  
print(addition_result) # Subtraction  
sub_result = matrix_a - matrix_b  
print("Subtraction (A - B):")  
print(sub_result) # Multiplication  
(element-wise)  
elemental_multi_result = matrix_a *  
matrix_b  
print("Element-wise Multiplication (A  
* B):") print(elemental_multi_result)  
# Matrix multiplication (dot product)  
matrix_dotproduct_result =  
np.dot(matrix_a , matrix_b ) print("A  
dot B:")  
print(matrix_dotproduct_result)  
# Transpose transpose_a  
= matrix_a.T  
print("Transpose of A:")  
print(transpose_a)
```

output :

10

```
print(sub_result)
```

Average time

0.018 s

18.25 ms

Maximum time

0.021 s

21.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

21 ms

Debug

Expected output

Actual output

Enter Matrix A:

Enter Matrix A:

1 2 3

1 2 3

4 5 6

4 5 6

7 8 9

7 8 9

Enter Matrix B:

Enter Matrix B:

1 1 1

1 1 1

Terminal

Test cases

3.2.2

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code :

```
import numpy as np #Input
```

```
matrices print("Enter
```

```
Array1:") arr1 =
```

```
np.array([list(map(int,
```

```
input().split())) for i in
```

```
range(3)])
```

```
print("Enter Array2:") arr2 = np.array([list(map(int,
input().split())) for i in range(3)])
```

```
#Perform horizontal stacking (hstack)
```

```
h_stack = np.hstack((arr1, arr2))
```

```
print("Horizontal Stack:")
```

```
print(h_stack)
```

```
#Perform vertical stacking (vstack)
```

```
v_stack = np.vstack((arr1, arr2))
```

```
print("Vertical Stack:")
```

```
print(v_stack) output :
```

The screenshot displays a code editor with the following Python code:

```
17 # Perform vertical stacking (vstack)
18 v_stack = np.vstack((arr1, arr2))
```

Below the code, the execution results are shown:

- Average time:** 0.024 s (24.25 ms)
- Maximum time:** 0.036 s (36.00 ms)
- Test Results:** 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1 (36 ms):

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
4 5 6	4 5 6

At the bottom, there are tabs for "Terminal" and "Test cases", and navigation buttons: "< Prev", "Reset", "Submit", and "Next >".

3.2.3

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code: import
numpy as np

```
#Take user input for the start, stop, and step of the sequence
```

```
start = int(input()) stop = int(input()) step = int(input())
```

```
#Generate the sequence using np.arange()
```

```
sequence = np.arange(start, stop, step) #
```

```
Print the generated sequence
```

```
print(sequence)
```

output:

Average time
0.014 s
14.25 ms

Maximum time
0.026 s
26.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 **26 ms** **Debug**

Expected output:

3

10

2

[3, 5, 7, 9]

Actual output:

3

10

2

[3, 5, 7, 9]

✓ Test case 2 **0 ms**

3.2.4

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code :

```
import numpy as np
```

```
def array_operations(A, B):
```

```
    #Convert A and B to NumPy arrays
```

```
    A= np.array(A)
```

```
    B= np.array(B)
```

```
# Arithmetic Operations
```

```
sum_result = A + B
```

```
diff_result = A - B
```

```
prod_result = A * B
```

```
# Statistical Operations
```

```
mean_A = np.mean(A)
```

```
median_A = np.median(A)
```

```
std_dev_A = np.std(A)
```

```
# Bitwise Operations
```

```
and_result = A & B
```

```
or_result = A | B
```

```
xor_result = A ^ B
```

```
#Output results with one space between each element print("Element-wise
```

```
Sum:", ' '.join(map(str, sum_result))) print("Element-wise Difference:",
```

```
' '.join(map(str, diff_result))) print("Element-wise Product:", ' '
```

```
'.join(map(str, prod_result))) print(f"Mean of A: {mean_A}")
```

```
print(f"Median of A: {median_A}") print(f"Standard Deviation of A:
```

```
{std_dev_A}")
```

```
print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
```

```
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A=list(map(int, input().split())) # Elements of array A
```

```
B=list(map(int, input().split())) # Elements of array B
```

array_operations(A, B) output:

```
26 print("Element-wise Sum:", ' '.join(map(str, sum_result)))
27 print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

Average time: **0.029 s** 29.00 ms
Maximum time: **0.071 s** 71.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 **71 ms** **Debug**

Expected output	Actual output
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4	Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32	Element-wise Product: 5 12 21 32
Mean of A: 2.5	Mean of A: 2.5

Terminal Test cases

3.2.5 The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: **<original_array>**

View array: **<view_array>**

- After modifying the copy:

Original array after modifying copy: **<original_array>**

Copy array: **<copy_array>** Code

and output :

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)
```

Average time
0.009 s
9.25 ms

Maximum time
0.016 s
16.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 16 ms

Test case 2 16 ms

3.2.6

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where search_value appears in array1 and print these indices.
- 2. Counting:** Count how many times count_value appears in array1 and print the count.
- 3. Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- 4. Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

code and output :

arrayCpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 a=np.where(array1==search_value)[0]
13 print(a)
14 # Count occurrences in array1
15 b=np.count_nonzero(array1==count_value)
16 print(b)
17
18 # Broadcasting addition
19 c=array1 + broadcast_value
20 print(c)
21
22 # Sort the first array
23 d=np.sort(array1)
24 print(d)
```

Average time
0.026 s
24.00 ms

Maximum time
0.057 s
57.00 ms

2 out of 2 shown test cases passed
2 out of 2 hidden test cases passed

Test case 1 ✓ 🔍 🔍

Expected output
4 1 1 1 1 1 1
Value to search: 1
Value to count: 2
Value to add: 2
(4 1 2)
3

Actual output
4 1 1 1 1 1 1
Value to search: 1
Value to count: 2
Value to add: 2
(4 1 2)
3

Terminal

Test cases

3.2.7

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.

- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
Print Subject 1 marks in ascending order: Sort and print the marks of students in
- Subject 1 in ascending order.
Print students who scored between 50 and 90 in Subject 1: Display students who
- scored marks between 50 and 90 in Subject 1.
Find index positions of students who scored 79 in Subject 1: Identify the index
- positions of students who scored exactly 79 marks in Subject 1.
-

Note: Fill in the missing code to perform the above-mentioned operations.

Code : import
numpy as np

a= np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

#1. Print all student details print("All
student Details:\n", a) #2. print
total students

print("Total Students:",a.shape[0])

#3. Print all student Roll numbers
print("All Student Roll Nos", a[:,0])

#4. Print subject 1 marks
print("Subject 1 Marks", a[:,1])

#5. print minimum marks of Subject 2

```
print("Min marks in Subject 2", np.min(a[:,2] ))
```

#6. print maximum marks of Subject 3

```
print("Max marks in Subject 3", np.max(a[:,3])  
)
```

#7. Print All subject marks print("All

```
subject marks:", a[:,1:]      )
```

#8. print Total marks of students print("Total

```
Marks", np.sum(a[:,1:],axis=1)      )
```

#9. print average marks of each student

```
avg=np.mean(a[:,1:],axis=1)
```

```
print(np.round(avg,1))
```

#10. print average marks of each subject print("Average Marks of each subject",

```
np.mean(a[:,1:],axis=0)      )
```

#11. print average marks of S1 and S2 print("Average Marks of S1

```
and S2", np.mean(a[:,1:3],axis=0)    )
```

#12. print average marks of S1 and S3 print("Average Marks of S1 and

```
S3", np.mean(a[:,[1,3]],axis=0)      )
```

```
#13. print Roll number who got maximum marks in Subject 3
i=np.argmax(a[:,3]) print("Roll no who got maximum marks in
Subject 3", a[i,0] )
```

```
#14. print Roll number who got minimum marks in Subject 2
```

```
j=np.argmin(a[:,2]) print("Roll no who got minimum marks
```

```
in Subject 2", a[j,0] ) #15. print Roll number who got 24
```

```
marks in Subject 2
```

```
print(f"Roll no who got 24 marks in Subject 2 [{a[a[:,2]==24,0]}]" )
```

```
#16. print count of students who got marks in Subject 1 < 40 print("Count of students
who got marks in Subject 1 < 40", np.sum(a[:,1]<40) )
```

```
#17. print count of students who got marks in Subject 2 > 90 print("Count of students
who got marks in Subject 2 > 90:", np.sum(a[:,2]>90) )
```

```
Count=[len(np.argwhere(a[:,1]>=90)),len(np.argwhere(a[:,2]>=90)),len(np.argwhere(a[:,3]>=
90))]
```

```
count=np.array(Count)
```

```
# 18. print count of students in each subject who got marks >= 90
```

```
print("Count of students in each subject who got marks >= 90:", count )
```

```
#19. print count of subjects in which each student got marks >= 90 print("Roll no:", a[:,0]
) print("Count of subjects in which student got marks >= 90:", np.sum(a[:,1:]>=90,axis=1)
)
```

#20. Print S1 marks in ascending order print(np.sort(a[:,1]))

print(a[(a[:,1]>=50)

& (a[:,1]<=90)]) print(a[(a[:,1]>=45) &

(a[:,1]<=90)]) k=np.where(a[:,1]==79)

print(k)

output :

```
12 print('All Student Roll nos :',a[:,0])
13
```

Average time: 0.074 s
74.00 ms

Maximum time: 0.074 s
74.00 ms

1 out of 1 shown test case(s) passed

Test case 1 4ms Debug

Expected output	Actual output
All student Details:	All student Details:
[[301, 67, 77, 88]]	[[301, 67, 77, 88]]
[302, 78, 88, 77]	[302, 78, 88, 77]
[303, 45, 56, 89]	[303, 45, 56, 89]
[304, 88, 98, 45]	[304, 88, 98, 45]
[305, 78, 88, 99]	[305, 78, 88, 99]

Terminal Test cases